

Crypto Echo - TCP Client-Server with Encryption

Assignment Overview

This assignment involves implementing a TCP client-server application that supports both plaintext and encrypted message communication. You will implement the socket programming and network protocol while using a provided cryptographic library for encryption operations.

Assignment Components: 1. **Programming (85 points):** Implement the TCP client and server 2. **Written Questions (15 points):** Answer 5 concept questions in `questions.md` 3. **Extra Credit (10 points):** Optional digital signatures feature

Learning Objectives: - Master TCP socket programming (socket, bind, listen, accept, connect, send, recv) - Implement a custom network protocol with Protocol Data Units (PDUs) - Handle key exchange for encrypted communication - Manage client-server state and bidirectional communication - Practice error handling and graceful shutdown procedures - Understand fundamental networking design decisions (stateful protocols, message framing, etc.)

What You're Implementing

The application consists of two main components:

Server (`crypto-server.c`)

- Creates a TCP socket and listens for incoming connections
- Accepts multiple clients sequentially (one at a time)
- Generates encryption keys upon client request
- Echoes received messages back to the client (with “echo” prefix)
- Handles both plaintext and encrypted messages
- Responds to shutdown commands

Client (`crypto-client.c`)

- Connects to the server via TCP
- Provides an interactive command-line interface
- Sends various message types (plaintext, encrypted, commands)
- Receives and displays server responses
- Manages encryption keys for the session

Building the Application

```
make clean
make
```

This will compile `crypto-echo` which can run as either a client or server.

Running the Application

Starting the Server

```
./crypto-echo --server
```

Options:

```
./crypto-echo --server --port 8080          # Use different port
./crypto-echo --server --addr 192.168.1.100  # Bind to specific IP
```

Default Settings: - Port: 1234 - Address: 0.0.0.0 (all interfaces)

The server will display:

```
Starting TCP server: binding to 0.0.0.0:1234
Server listening on 0.0.0.0:1234
Waiting for client connection...
```

Starting the Client

```
./crypto-echo --client
```

Options:

```
./crypto-echo --client --port 8080          # Connect to different port
./crypto-echo --client --addr 192.168.1.100 # Connect to specific server
```

Default Settings: - Port: 1234 - Address: 127.0.0.1 (localhost)

The client will display:

```
Starting TCP client: connecting to 127.0.0.1:1234
Connected to server 127.0.0.1:1234
>
```

Client Interface Commands

The client provides an interactive command-line interface with the following commands:

Basic Commands

Command	Description	Example
<text>	Send plaintext message	Hello World
!<text>	Send encrypted message (requires key exchange first)	!Secret message
#	Request encryption key exchange	#
?	Display help information	?
-	Exit the client	-


```
PDU Info:  
Type: DATA  
Direction: REQUEST  
Payload Length: 15 bytes  
Payload (plaintext): Basic Echo Demo  
>>>>>>>>>> END REQUEST >>>>>>>>>>
```

```
PDU Info:  
Type: DATA  
Direction: RESPONSE  
Payload Length: 20 bytes  
Payload (plaintext): echo Basic Echo Demo  
<<<<<<<<<< END RESPONSE <<<<<<<<<<
```

[illegible]

<<<<<<<<<< RESPONSE <<<<<<<<<<

[illegible]

Type	Value	Description
MSG_KEY_EXCHANGE	1	Request/response for encryption keys
MSG_DATA	2	Plaintext message
MSG_ENCRYPTED_DATA	3	Encrypted message
MSG_DIG_SIGNATURE	4	Message with digital signature (extra credit)
MSG_CMD_CLIENT_STOP	5	Client exit command
MSG_CMD_SERVER_STOP	6	Server shutdown command

Typical Communication Flow

1. **Client connects** to server via TCP
2. **Client sends** MSG_KEY_EXCHANGE request
3. **Server responds** with MSG_KEY_EXCHANGE containing client's encryption key
4. **Client stores** the key for the session
5. **Client sends** MSG_DATA or MSG_ENCRYPTED_DATA messages
6. **Server processes** and responds with echoed message
7. **Client sends** exit command when done

Important Notes

Character Restrictions for Encryption

Encrypted messages can only contain these 64 characters: - Uppercase letters: A-Z - Lowercase letters: a-z - Digits: 0-9 - Space and comma: ,

Other punctuation (., !, ?, etc.) will cause encryption errors.

Key Management

- Keys are generated by the **server** when requested
- Keys are session-specific (new keys for each connection)
- The server keeps two keys: one for itself, one for the client
- The server sends the **client's key** during key exchange
- Both parties use their respective keys throughout the session

Debugging

The `print_msg_info()` function (from `crypto-lib`) displays detailed PDU information, including: - Message type and direction - Payload length - Plaintext or encrypted payload content - Decrypted content (when possible)

Use this liberally during development!

Files You Need to Implement

Required Implementation

1. **crypto-client.c**
 - `start_client()` - Connect to server and manage communication
 - Additional helper functions as needed
2. **crypto-server.c**
 - `start_server()` - Create server socket, accept connections, handle clients
 - Additional helper functions as needed

Provided Files (Do NOT Modify)

- `crypto-lib.c` / `crypto-lib.h` - Encryption library
- `crypto-echo.c` / `crypto-echo.h` - Main program and command-line parsing
- `protocol.h` - Protocol constants and structures
- `makefile` - Build configuration

Testing Strategy

1. **Start Simple**
 - Get server listening and accepting connections
 - Get client connecting to server
 - Send/receive plaintext messages
2. **Add Key Exchange**
 - Client sends `#` command
 - Server generates keys with `gen_key_pair()`
 - Server sends client's key back
 - Client stores the key
3. **Test Encryption**
 - Client sends `!message`
 - Verify encryption with `encrypt_string()`
 - Server decrypts with `decrypt_string()`
 - Server echoes and re-encrypts
 - Client decrypts and displays
4. **Test Exit Paths**
 - Test - (client exit only)
 - Test = (client and server exit)
 - Test unexpected disconnection

Common Issues and Solutions

“**Connection refused**” - Ensure server is running before starting client - Check port numbers match - Verify firewall isn't blocking the port

“**Address already in use**” - Previous server instance still running - Use `SO_REUSEADDR` socket option (helpful during development) - Wait a minute or use

a different port

“No session key established” - Must send `#` command before `!message` - Check that key exchange completed successfully - Verify key isn't `NULL_CRYPT0_KEY` (0xFFFF)

“Invalid character” errors - Encrypted messages have character restrictions - Only use: A-Z a-z 0-9 , and space - Plaintext messages don't have this limitation

Extra Credit: Digital Signatures (10 points)

Implement the `$` command to send messages with digital signatures. This demonstrates how to verify message integrity and authenticity.

Overview

A digital signature allows the receiver to verify: 1. The message hasn't been tampered with 2. The message came from the expected sender

Implementation Approach

Hash Function (Keep It Simple) Create a simple one-byte hash of the message:

```
uint8_t compute_hash(const char *message, size_t len) {
    uint8_t hash = 0;
    for (size_t i = 0; i < len; i++) {
        hash ^= message[i]; // XOR all bytes
        hash = (hash << 1) | (hash >> 7); // Rotate left
    }
    return hash;
}
```

This isn't cryptographically secure, but it demonstrates the concept!

Client Side - Sending Signed Message When the user types `$Hello World`:

1. **Extract the message:** "Hello World"
2. **Compute hash:** `hash = compute_hash("Hello World", 11)`
3. **Encrypt the hash:** `encrypt(session_key, &encrypted_hash, &hash, 1)`
4. **Build the payload:** `[encrypted_hash_byte][plaintext_message]`
5. **Create PDU:**

```
pdu->header.msg_type = MSG_DIG_SIGNATURE;
pdu->header.direction = DIR_REQUEST;
pdu->payload[0] = encrypted_hash;
```

```
memcpy(&pdu->payload[1], message, message_len);
pdu->header.payload_len = 1 + message_len;
```

6. Send to server

Server Side - Verifying and Responding When receiving MSG_DIG_SIGNATURE:

1. **Extract encrypted signature:** encrypted_sig = request->payload[0]
2. **Extract message:** message = &request->payload[1]
3. **Decrypt signature:** decrypt(server_key, &received_hash, &encrypted_sig, 1)
4. **Compute hash of message:** computed_hash = compute_hash(message, len-1)
5. **Verify:**

```
if (received_hash != computed_hash) {
    // Signature verification failed!
    printf("[ERROR] Digital signature verification failed!\n");
    // Send error response
} else {
    // Signature valid, process message
}
```
6. **Create response:** "echo <original message>"
7. **Sign the response:**
 - Compute hash of response
 - Encrypt the hash
 - Send: [encrypted_hash][response_text]
8. **Send back to client**

Client Side - Verifying Server Response When receiving MSG_DIG_SIGNATURE response:

1. **Extract encrypted signature:** encrypted_sig = response->payload[0]
2. **Extract message:** message = &response->payload[1]
3. **Decrypt signature:** decrypt(session_key, &received_hash, &encrypted_sig, 1)
4. **Compute hash of message:** computed_hash = compute_hash(message, len-1)
5. **Verify:**

```
if (received_hash != computed_hash) {
    printf("[WARNING] Server response signature invalid!\n");
} else {
    printf("✓ Server response verified!\n");
    printf("Message: %.*s\n", message_len, message);
}
```

Implementation Tips

1. **Update `get_command()`:** Make the '\$' case return `CMD_EXECUTE` instead of `CMD_NO_EXEC`
2. **Create helper function:** `int add_signature(crypto_key_t key, uint8_t *payload, const char *msg, size_t msg_len)`
3. **Create helper function:** `int verify_signature(crypto_key_t key, const uint8_t *payload, size_t total_len)`
4. **Handle in both client and server:** Both need to sign outgoing and verify incoming
5. **Test thoroughly:** Try modifying the message or signature to verify detection

Testing Digital Signatures

[illegible]

What Makes This Extra Credit?

- Demonstrates understanding of cryptographic signatures

- Requires careful payload construction
- Involves encryption/decryption of non-text data (the hash byte)
- Shows integrity verification in action
- More complex protocol handling

Submission

Document your implementation: - Explain your hash function choice - Show example interactions - Discuss what happens when signature verification fails - Describe how this prevents tampering

Resources

- **README.md** - This file; assignment overview and requirements
- **questions.md** - Five concept questions (15 points of your grade)
- **crypto.md** - Detailed explanation of the encryption algorithm
- **crypto-lib.h** - Comprehensive API documentation for crypto functions
- **Man pages:** `man socket`, `man bind`, `man listen`, `man accept`, `man connect`, `man send`, `man recv`

Grading Criteria

Total Points: 100

Programming Implementation (85 points)

Component	Points
Server socket setup and listening	15
Client connection establishment	10
Plaintext message handling (send/receive/echo)	15
Key exchange implementation	20
Encrypted message handling (encrypt/decrypt)	20
Exit command handling (- and =)	5
Programming Subtotal	85

Concept Questions (15 points)

Component	Points
Five written questions (3 points each)	15
Questions Subtotal	15

Extra Credit (Optional)

Component	Points
Digital signatures implementation (\$ command)	+10

Maximum Possible Score: 110 points (100 base + 10 extra credit)

Submission Requirements

Submit the following files:

Required Files (Programming - 85 points)

1. `crypto-client.c` - Your client implementation
2. `crypto-server.c` - Your server implementation
3. `crypto-client.h` - Client header (if you added functions)
4. `crypto-server.h` - Server header (if you added functions)

Required Files (Questions - 15 points)

5. `lastname_firstname_answers.md` (or `.txt` or `.pdf`) - Your answers to the 5 concept questions from `questions.md`

Optional (Extra Credit - +10 points)

6. If implementing digital signatures, ensure your code handles the \$ command

Do NOT Submit

- `crypto-lib.c` / `crypto-lib.h` (provided library)
- `crypto-echo.c` / `crypto-echo.h` (provided main)
- `protocol.h` (provided)
- `makefile` (provided)
- Compiled binaries

Note: Your code must compile with the provided files using `make`. Test this before submitting!

Academic Integrity

This is an individual assignment. You may discuss concepts with classmates, but all code must be your own. The provided crypto library and scaffold code are starter materials - your task is to implement the networking components.

Good luck! Focus on getting the basic socket operations working first, then add encryption features incrementally. Use `print_msg_info()` extensively for debugging!